

Offline Gesture Recognition Application

Version 1

System Architecture Design (SAD)

At a glance	
Platform	Cross-platform desktop application for Windows, macOS, and Linux
Technology	Python, PySide6 (GUI), OpenCV (camera capture and image processing)
Function	Real-time recognition of ten predefined hand gestures from a local camera
Operation	Fully offline; frames and recognition data stay in memory
Architecture	Modular, single-process, multithreaded

This document describes the high-level software architecture of the application. The architecture diagram is identified as Figure SAD-01.

1. Purpose and Scope

This document defines the high-level software architecture for Version 1 of a cross-platform desktop application that recognizes ten predefined hand gestures in real time using a local camera. The application is developed in Python, uses PySide6 for the graphical user interface (GUI), and uses OpenCV for camera capture and image processing.

The system must operate entirely offline on Windows, macOS, and Linux. Camera frames and recognition data are processed in memory and are not collected, persistently stored, or transmitted. Local configuration and diagnostic logs are permitted, provided they exclude camera frames and sensitive user data.

2. Architectural Overview

The application follows a modular, single-process, multithreaded architecture. Components communicate through defined interfaces and shared domain data contracts. The Qt main thread manages the GUI, while camera capture and computationally intensive vision processing execute in worker threads.

The recognition engine uses pretrained hand landmark detection combined with a hybrid recognition layer. Geometric rules handle suitable static poses, temporal state machines detect directional movements, and machine-learning classifiers are used where appropriate. A

centralized decision and event management component validates recognition results, applies confidence thresholds, and suppresses duplicate events.

All models and runtime dependencies must be available locally. Runtime network access and automatic model downloads are excluded from the design.

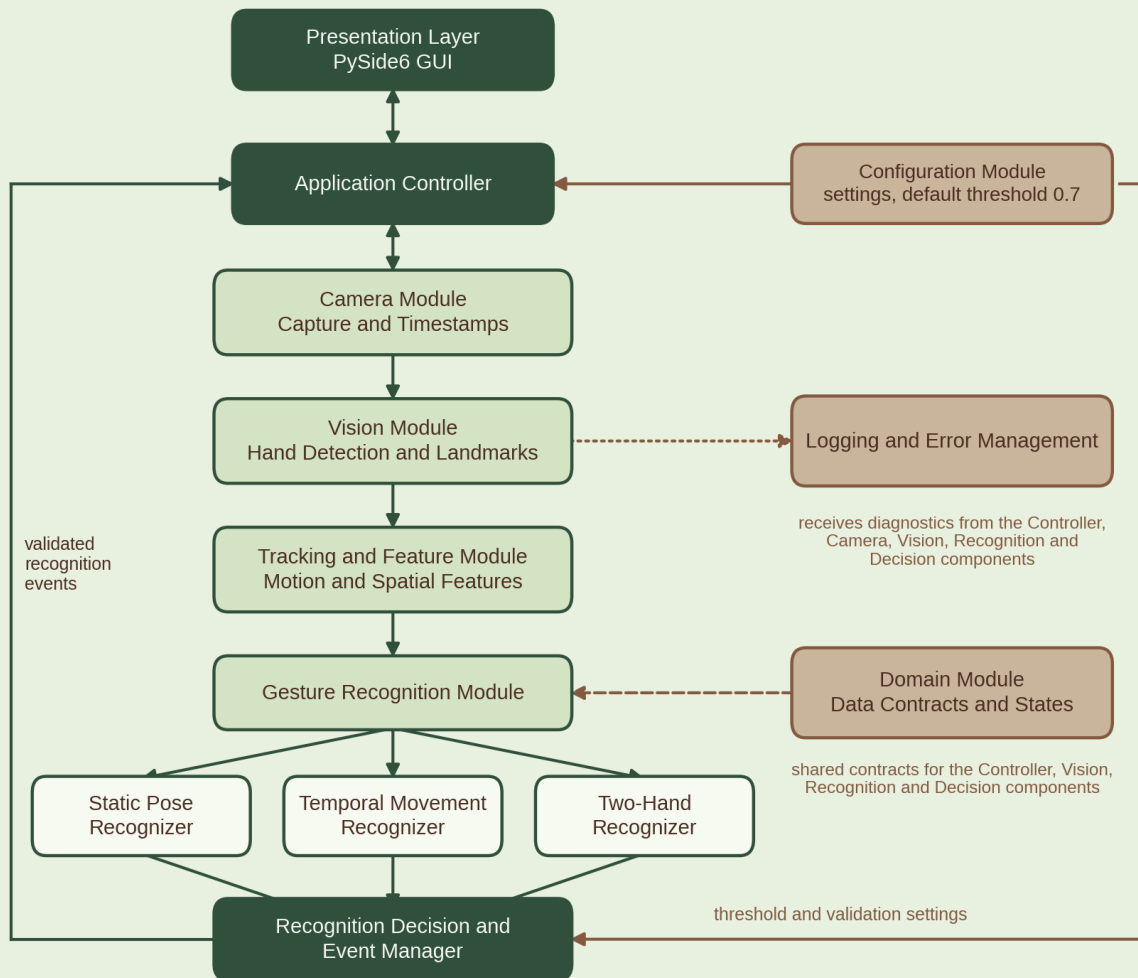


Figure SAD-01: High-level architecture. Processing flows from camera to decision manager; supporting modules are shown in tan.

3. Major Components and Responsibilities

Component	Responsibility
Presentation Layer	Displays the camera preview, bounding boxes, recognized gesture labels, and five indicators: Last Gesture, Current Gesture, Confidence, Camera FPS, and Processing FPS.
Application Controller	Coordinates application startup, shutdown, worker lifecycle, component communication, and recognition state updates.

Component	Responsibility
Camera Module	Initializes the camera, captures timestamped frames, and reports device or capture failures.
Vision Module	Performs hand detection, landmark extraction, and required image preprocessing.
Tracking and Feature Module	Maintains hand tracking, derives finger-pose features, tracks motion history, and calculates two-hand spatial relationships.
Gesture Recognition Module	Provides replaceable recognition strategies for static poses, temporal movements, and two-hand gestures.
Recognition Decision and Event Manager	Applies confidence thresholds, validates recognition criteria, manages re-arming and duplicate suppression, and emits recognition events.
Configuration Module	Loads, validates, and supplies application settings, including the default confidence threshold of 0.7.
Logging and Error Management Module	Records operational events, diagnostic information, exceptions, and recovery actions without sensitive image data.
Domain Module	Defines shared data contracts, gesture identifiers, recognition results, application states, and error types.

Recognized gestures

The ten externally recognized gesture labels are:

- Swipe Left
- Swipe Right
- Peace Sign
- Close Hand
- Open Hand
- Wave Left with Index Finger Raised
- Wave Right with Index Finger Raised
- Wave Left with Index and Middle Fingers Raised
- Wave Right with Index and Middle Fingers Raised
- Join Both Hands in a Clap/Prayer Gesture

Prayer and clap share one label

The static prayer pose and the dynamic clap motion must produce the same external gesture label. Internal recognition strategies may distinguish these patterns while exposing a consistent result through the recognition interface.

4. Runtime Data Flow

1. The Application Controller initializes the configuration, loads local recognition models, and starts the camera and processing workers.
2. The Camera Module captures and timestamps frames. A bounded buffer uses a latest-frame-wins policy to discard obsolete frames when processing cannot keep pace with capture.
3. The Vision Module detects hands and extracts landmarks. The Tracking and Feature Module derives pose, motion, and two-hand relationship features.
4. The Gesture Recognition Module evaluates applicable static, temporal, and two-hand strategies. Candidate results include a gesture identifier, confidence score, and relevant tracking information.
5. The Recognition Decision and Event Manager validates the candidate against gesture-specific criteria and the configured confidence threshold (default 0.7), with additional temporal validation where required.
6. Upon a valid recognition, the manager emits an event to the Application Controller, which updates the GUI through thread-safe communication. Recognized hands receive green bounding boxes and gesture labels; hands without an active recognized gesture receive grey bounding boxes.
7. The GUI updates its five indicators using the latest valid recognition and performance measurements. Re-arming, cooldown, or gesture-specific state management prevents repeated events for sustained gestures.

The GUI must remain responsive throughout processing. UI updates occur exclusively on the Qt main thread.

5. Key Architectural Decisions and Trade-offs

Modular, single-process execution. A single process simplifies deployment and component integration. Worker threads isolate blocking capture and expensive processing from the GUI, although Python runtime constraints and model-library behavior must be considered when selecting concurrency mechanisms.

Pretrained and hybrid recognition. Pretrained landmark detection reduces the need to develop a hand detector from scratch. Geometric rules are interpretable and efficient for suitable static poses; temporal state machines represent directional movements; classifiers can accommodate patterns that are difficult to express reliably through rules alone. This hybrid design improves flexibility but introduces model dependencies and additional validation requirements.

Replaceable recognition interface. The controller communicates with the recognition engine through a stable interface, allowing models and classification strategies to change without redesigning the camera or presentation components.

Latest-frame-wins buffering. A bounded frame queue prioritizes current visual information and limits latency and memory growth. Frames may be dropped under load, trading processing completeness for responsiveness.

Confidence and event control. A configurable default threshold of 0.7 provides a common baseline. Gesture-specific temporal criteria and duplicate suppression address false activations and repeated events. Confidence calibration and gesture-specific criteria require empirical validation.

Offline operation. Bundled or locally installed models and dependencies avoid runtime network requirements and reduce data exposure. The trade-off is that dependency installation, model availability, and version management must be handled explicitly during development and deployment.

6. Error Handling, Logging, and Privacy

Errors are handled at component boundaries and reported to the Application Controller for coordinated recovery or controlled shutdown. Expected failure cases include unavailable cameras, capture interruptions, model-loading failures, inference exceptions, invalid configuration, worker-thread failures, and shutdown errors.

Recoverable failures should trigger appropriate retries or component restarts where safe. Unrecoverable failures should produce a clear user-facing error and release camera and worker resources cleanly. Logging records lifecycle events, error categories, component status, and recovery actions while excluding image data, landmarks that could expose user activity, and other sensitive information.

Camera frames and transient recognition data remain in memory. No cloud services, remote APIs, telemetry, databases, or network-dependent runtime features are introduced.

7. Extensibility and Testing

Clear interfaces and shared domain contracts support the addition of gestures, replacement of recognition models, and adjustment of recognition policies without coupling those changes to the GUI or camera implementation.

Testing includes unit tests for geometric features, recognition state machines, confidence enforcement, event suppression, and configuration validation; integration tests for camera-to-recognition data flow and UI event delivery; and hardware-dependent tests for camera access, performance, and cross-platform behavior. Model-loading failures, dropped frames, concurrent shutdown, and worker exceptions must also be tested.

Established decisions and open items

The architecture and technology stack are established decisions. The exact landmark model, classifier selection, gesture-specific temporal parameters, thread implementation, camera selection policy, and performance targets remain implementation decisions to be finalized through prototyping and testing.